

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НТУ «Дніпровська політехніка»

Інститут електроенергетики

Факультет інформаційної технології

Кафедра САіУ

ЗВІТ

з лабораторної роботи

дисципліни «Веб-застосунки з Java/Spring»

Виконала: ст. гр. 122м-24-2

Гродська Марія Євгенівна

Перевірив:

доц. Мінєєв О.С

Дніпро

2025

Завдання: Розробити програму мовою Java на базі фреймворку Spring boot. Спілкування клієнта/сервера реалізувати через REST.

- Додаток повинен вивантажувати звіт у форматі Excel
- Додаток повинен парсити інформацію з інтернету
- Додаток також повинен брати інформацію по відкритому Rest API
- Додаток повинен мати Front End частину

Результатом роботи буде додаток SPRING BOOT (у вигляді jar) який запускається bat(перевіряю на Windows) файлом формату “java -jar *.jar”

Хід роботи

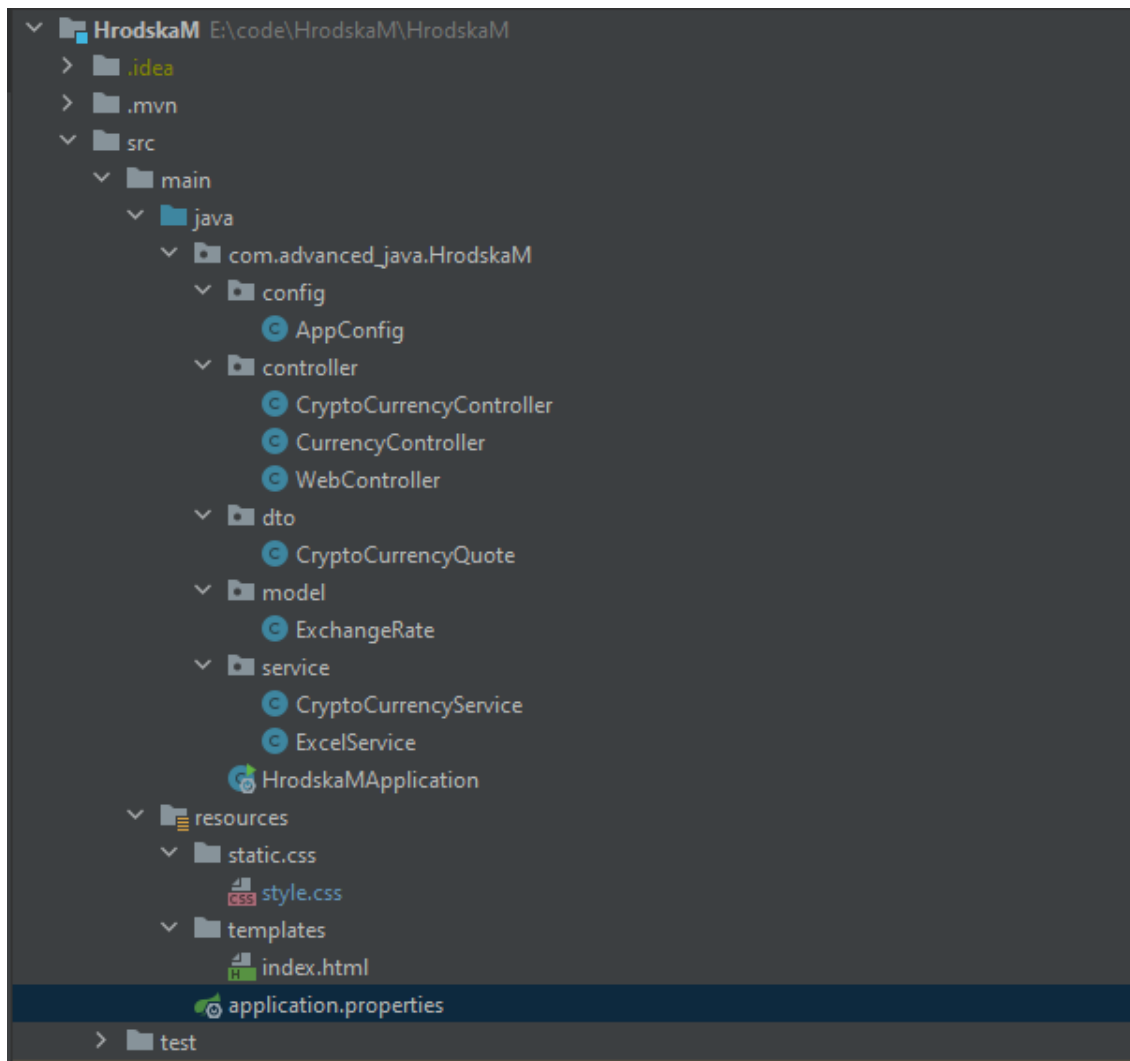


Рис 1. Файлова структура проекту

Опис реалізованого застосунку

Мета застосунку — створити простий веб-інструмент, який дозволяє швидко дізнаватися поточний курс Bitcoin, здійснювати пошук криптовалют за назвою, а також вивантажувати список знайдених криптовалют у форматі Excel.

Основний клас `HrodskApplication` відповідає за запуск Spring Boot-додатку. Конфігураційний файл `pom.xml` містить усі необхідні залежності, які використовуються в проєкті, зокрема Spring Boot, Jsoup, Apache POI та Thymeleaf.

Клас `AppConfig` створює `RestTemplate`, який використовується для виконання HTTP-запитів до зовнішніх API. Контролер `CurrencyController` обробляє запит для отримання поточного курсу Bitcoin, використовуючи сервіс `CryptoCurrencyService`, який надсилає запит до API біржі.

Контролер `CryptoCurrencyController` дозволяє здійснювати пошук криптовалют за назвою. Для цього використовується `CryptoCurrencyService`, який парсить HTML сторінки результатів з допомогою бібліотеки Jsoup. Також цей контролер надає можливість експорту знайдених криптовалют у файл Excel за допомогою `ExcelService`, який формує таблицю з даними за допомогою Apache POI.

Клас `WebController` відповідає за відображення головної HTML-сторінки `index.html`. Інтерфейс дає змогу переглядати курс Bitcoin, виконувати пошук криптовалют і завантажувати таблицю з результатами.

Модель `ExchangeRate` описує структуру валютних даних, а `CryptoCurrencyQuote` — криптовалютні котирування з часом, ціною та назвою криптовалюти.

Користувач може відкривати інтерфейс за адресою:

<http://localhost:8081> — після запуску jar-файлу через `.bat`.

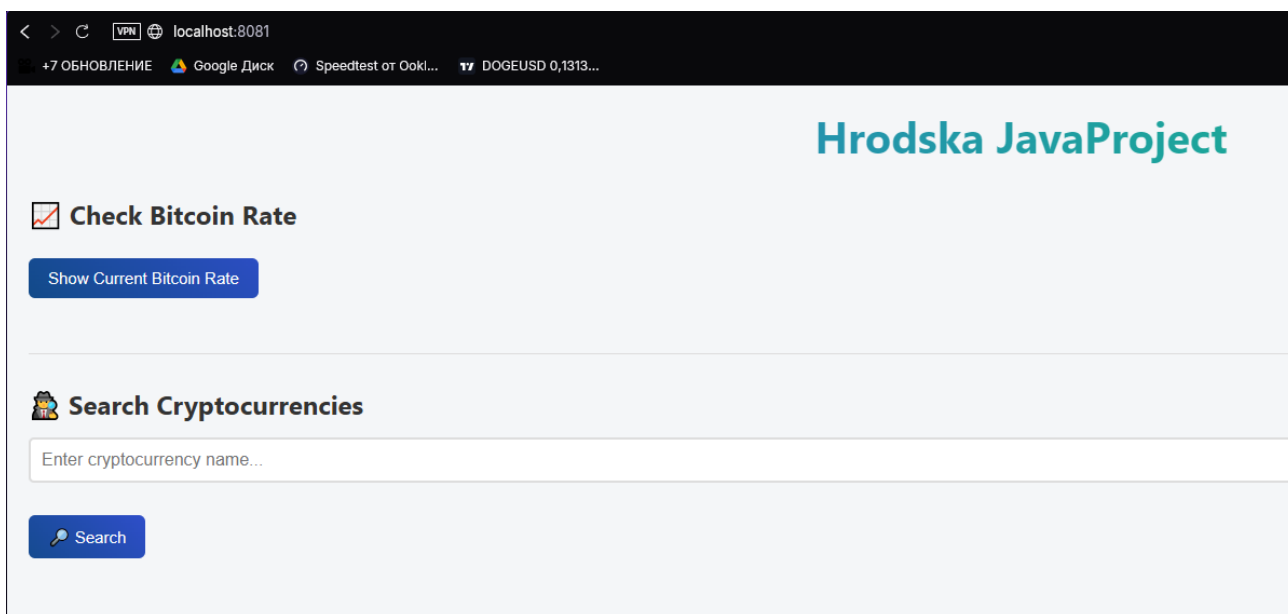


Рис 2. Інтерфейс програми

Користувач натискає кнопку Show Current Bitcoin Rate, і програма відображає актуальний курс Bitcoin.

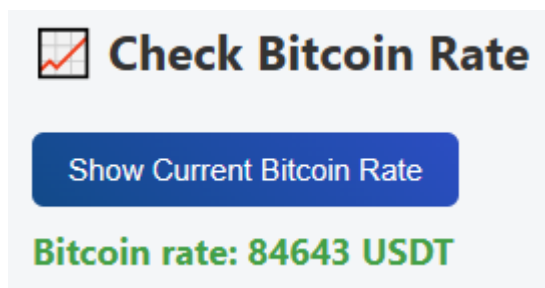


Рис 3. Актуальний курс Bitcoin

Користувач вводить назву криптовалюти у відповідне поле, натискає кнопку Search, після чого програма виконує пошук та відображає таблицю з актуальними даними: назва криптовалюти, її ціна в доларах, дата та час отримання інформації.

Search Cryptocurrencies			
solana			
Search Download Excel File			
Currency	Price (USD)	Date	Time
SOLANA	135.84	16 березня 2025 р.	02:00
SOLANA	126.17	17 березня 2025 р.	02:00
SOLANA	128.28	18 березня 2025 р.	02:00
SOLANA	125.34	19 березня 2025 р.	02:00
SOLANA	135.62	20 березня 2025 р.	02:00
SOLANA	127.68	21 березня 2025 р.	02:00
SOLANA	128.24	22 березня 2025 р.	02:00
SOLANA	128.39	23 березня 2025 р.	02:00
SOLANA	132.10	24 березня 2025 р.	02:00
SOLANA	140.58	25 березня 2025 р.	02:00
SOLANA	143.93	26 березня 2025 р.	02:00

Рис 4. Таблиця пошуку

Після виконання пошуку користувач може натиснути кнопку "Download Excel File", щоб зберегти результати у вигляді Excel-документа. У відповідь програма генерує файл, який містить знайдену криптовалюту з відповідною інформацією — назвою, актуальною ціною, датою та часом — і надає його для завантаження на комп'ютер користувача.

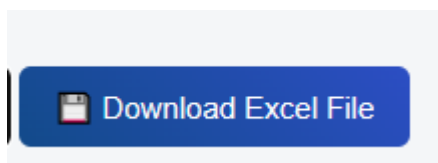


Рис 5. Кнопка завантаження

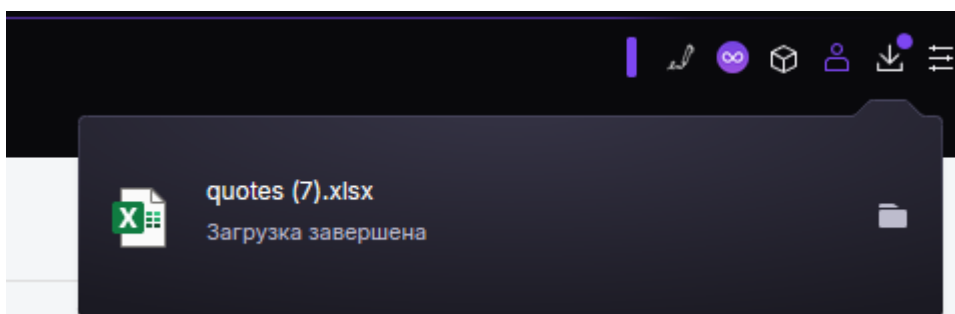


Рис 6. Завантаження звіту

	A	B	C	D	E
1	Currency	Price (USD)	Timestamp		
2	solana	135,8380809	2025-03-16 02:00:00		
3	solana	126,1651453	2025-03-17 02:00:00		
4	solana	128,2798095	2025-03-18 02:00:00		
5	solana	125,3440601	2025-03-19 02:00:00		
6	solana	135,6227617	2025-03-20 02:00:00		
7	solana	127,6762642	2025-03-21 02:00:00		
8	solana	128,2397382	2025-03-22 02:00:00		
9	solana	128,3918739	2025-03-23 02:00:00		
10	solana	132,1010687	2025-03-24 02:00:00		
11	solana	140,5793688	2025-03-25 02:00:00		
12	solana	143,9330513	2025-03-26 02:00:00		
13	solana	137,0933425	2025-03-27 02:00:00		
14	solana	138,3789793	2025-03-28 02:00:00		
15	solana	129,4462168	2025-03-29 02:00:00		
16	solana	124,5018966	2025-03-30 02:00:00		
17	solana	124,5806499	2025-03-31 03:00:00		
18	solana	124,8661784	2025-04-01 03:00:00		
19	solana	126,8069368	2025-04-02 03:00:00		
20	solana	118,0855059	2025-04-03 03:00:00		
21	solana	117,1929519	2025-04-04 03:00:00		
22	solana	122,7130747	2025-04-05 03:00:00		
23	solana	120,1683821	2025-04-06 03:00:00		
24	solana	105,7651501	2025-04-07 03:00:00		
25	solana	106,7967654	2025-04-08 03:00:00		
26	solana	105,4865468	2025-04-09 03:00:00		
27	solana	118,9628806	2025-04-10 03:00:00		
28	solana	112,8861685	2025-04-11 03:00:00		
29	solana	121,3924954	2025-04-12 03:00:00		
30	solana	132,1543853	2025-04-13 03:00:00		
31	solana	128,2347433	2025-04-14 03:00:00		
32	solana	129,1871111	2025-04-15 01:20:41		
33					

Рис 7. Excel файл

Для зручності запуску застосунку створено батч-файл (batch file), який автоматизує процес відкриття веб-інтерфейсу та запуску Spring Boot застосунку.

```
C:\WINDOWS\system32\cmd.exe
tory interfaces.
2025-04-15T01:36:25.527+03:00 INFO 24568 --- [Hrodskam] [ main] o.s.cloud.context.scope.GenericScope : BeanFactory id=10fad0c-bab3-3a13-bb99-9e35ebb63ec
2025-04-15T01:36:26.175+03:00 INFO 24568 --- [Hrodskam] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8081 (http)
2025-04-15T01:36:26.191+03:00 INFO 24568 --- [Hrodskam] [ main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-04-15T01:36:26.192+03:00 INFO 24568 --- [Hrodskam] [ main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.39]
2025-04-15T01:36:26.228+03:00 INFO 24568 --- [Hrodskam] [ main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-04-15T01:36:26.229+03:00 INFO 24568 --- [Hrodskam] [ main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed
2025-04-15T01:36:26.393+03:00 INFO 24568 --- [Hrodskam] [ main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2025-04-15T01:36:26.610+03:00 INFO 24568 --- [Hrodskam] [ main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn0: url=jdbc:h2
412c-9d3b-8e74c58b3c69 user=SA
2025-04-15T01:36:26.612+03:00 INFO 24568 --- [Hrodskam] [ main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2025-04-15T01:36:26.690+03:00 INFO 24568 --- [Hrodskam] [ main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: d
2025-04-15T01:36:26.768+03:00 INFO 24568 --- [Hrodskam] [ main] org.hibernate.Version : HHH000412: Hibernate ORM core version 6.6.11.Final
2025-04-15T01:36:26.816+03:00 INFO 24568 --- [Hrodskam] [ main] o.h.c.internal.RegionFactoryInitiator : HHH00026: Second-level cache disabled
2025-04-15T01:36:27.135+03:00 INFO 24568 --- [Hrodskam] [ main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transf
2025-04-15T01:36:27.208+03:00 INFO 24568 --- [Hrodskam] [ main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 2.3.232
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-04-15T01:36:27.487+03:00 INFO 24568 --- [Hrodskam] [ main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hiberna
platform' to enable JTA platform integration)
2025-04-15T01:36:27.491+03:00 INFO 24568 --- [Hrodskam] [ main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persisten
2025-04-15T01:36:27.803+03:00 WARN 24568 --- [Hrodskam] [ main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. The
eries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2025-04-15T01:36:27.828+03:00 INFO 24568 --- [Hrodskam] [ main] o.s.b.a.w.s.WelcomePageHandlerMapping : Adding welcome page template: index
2025-04-15T01:36:28.238+03:00 INFO 24568 --- [Hrodskam] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8081 (http) with context pa
2025-04-15T01:36:28.258+03:00 INFO 24568 --- [Hrodskam] [ main] c.a.Hrodskam.HrodskamApplication : Started HrodskamApplication in 4.42 seconds (proce
1)
2025-04-15T01:36:28.456+03:00 INFO 24568 --- [Hrodskam] [nio-8081-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherS
2025-04-15T01:36:28.457+03:00 INFO 24568 --- [Hrodskam] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-04-15T01:36:28.459+03:00 INFO 24568 --- [Hrodskam] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
```

Рис 7. Запуск Start_Hrodskalab.bat

Код фрагменту:

```
@echo off
start http://localhost:8081
java -jar target\Hrodskam-0.0.1-SNAPSHOT.jar
pause
```

Висновок:

У ході лабораторної роботи було розроблено повнофункціональний вебзастосунок на базі Spring Boot, який реалізує інтеграцію з відкритими API для отримання актуального курсу криптовалют, а також функціонал пошуку та збору даних про криптовалюту. За допомогою бібліотеки Jsoup здійснюється парсинг результатів пошуку з вебсайтів, а за допомогою Apache POI забезпечено можливість експорту результатів у формат Excel. Застосунок використовує REST архітектуру для обміну даними між сервером і клієнтом, а також має простий і зручний фронтенд для взаємодії з користувачем. Робота над проєктом дозволила здобути практичні навички в роботі з Spring Boot, парсингу вебсторінок та інтеграції сторонніх бібліотек і API в Java-застосунки.

Лістинг програми

HrodskaMApplication.java

```
package com.advanced_java.HrodskaM;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HrodskaMApplication {
    public static void main(String[] args) {
        SpringApplication.run(HrodskaMApplication.class, args);
    }
}
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.advanced_java</groupId>
    <artifactId>HrodskaM</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>HrodskaM</name>
    <description>Hrodska Mariia JavaLab</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
        <spring-cloud.version>2024.0.1</spring-cloud.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.jsoup</groupId>
            <artifactId>jsoup</artifactId>
            <version>1.16.1</version>
        </dependency>
    </dependencies>
```



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.2.3</version>
</dependency>
<dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
  <version>3.1.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>4.0.1</version>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-xml</artifactId>
  <version>2.13.1</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
</dependencies>
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <version>3.2.5</version>
            <executions>
                <execution>
                    <goals>
                        <goal>repackage</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>

```

</project>

AppConfig.java

```

package com.advanced_java.HrodskaM.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

```

```
}  
}
```

CryptoCurrencyController.java

```
package com.advanced_java.HrodskaM.controller;  
import com.advanced_java.HrodskaM.dto.CryptoCurrencyQuote;  
import com.advanced_java.HrodskaM.service.CryptoCurrencyService;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestParam;  
import org.springframework.web.bind.annotation.RestController;  
  
import java.util.List;  
@RestController  
public class CryptoCurrencyController {  
    private final CryptoCurrencyService cryptoCurrencyService;  
    public CryptoCurrencyController(CryptoCurrencyService cryptoCurrencyService)  
    {  
        this.cryptoCurrencyService = cryptoCurrencyService;  
    }  
    @GetMapping("/get-bitcoin-rate")  
    public ResponseEntity<String> getBitcoinRate() {  
        String bitcoinRate = cryptoCurrencyService.getBitcoinRate(); // Get the  
Bitcoin rate  
        return ResponseEntity.ok(bitcoinRate);  
    }  
    @GetMapping("/parse-crypto")  
    public ResponseEntity<List<CryptoCurrencyQuote>> parseCrypto(@RequestParam  
String searchQuery) {  
        List<CryptoCurrencyQuote> cryptocurrencies =  
cryptoCurrencyService.getQuotes(searchQuery);  
        return ResponseEntity.ok(cryptocurrencies);  
    }  
}
```

CurrencyController.java

```
package com.advanced_java.HrodskaM.controller;  
import org.springframework.ui.Model;  
import com.advanced_java.HrodskaM.dto.CryptoCurrencyQuote;  
import com.advanced_java.HrodskaM.service.CryptoCurrencyService;  
import com.advanced_java.HrodskaM.service.ExcelService;  
import jakarta.servlet.http.HttpServletRequestResponse;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestParam;  
import org.springframework.web.bind.annotation.RestController;  
import java.util.List;  
@RestController  
public class CurrencyController {  
    @Autowired  
    private CryptoCurrencyService cryptoService;  
  
    @Autowired  
    private ExcelService excelService;  
  
    @GetMapping("/search")  
    public String searchCrypto(@RequestParam("name") String name, Model model) {  
        List<CryptoCurrencyQuote> quotes =  
cryptoService.getQuotes(name.toLowerCase());  
    }  
}
```

```

        model.addAttribute("quotes", quotes);
        model.addAttribute("currencyName", name);
        return "index";
    }
    @GetMapping("/download")
    public void downloadExcel(@RequestParam("name") String name,
        HttpServletResponse response) throws Exception {
        List<CryptoCurrencyQuote> quotes =
        cryptoService.getQuotes(name.toLowerCase());
        excelService.exportToExcel(quotes, response);
    }
}

```

WebController.java

```

package com.advanced_java.HrodskaM.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class WebController {

    @GetMapping("/")
    public String home() {
        return "index";
    }

}

```

CryptoCurrencyQuote.java

```

package com.advanced_java.HrodskaM.dto;
import java.time.LocalDateTime;
public class CryptoCurrencyQuote {
    private String currency;
    private double price;
    private LocalDateTime timestamp;
    public CryptoCurrencyQuote(String currency, double price, LocalDateTime
timestamp) {
        this.currency = currency;
        this.price = price;
        this.timestamp = timestamp;
    }
    public String getCurrency() {
        return currency;
    }
    public void setCurrency(String currency) {
        this.currency = currency;
    }
    public double getPrice() {
        return price;
    }
    public void setPrice(double price) {
        this.price = price;
    }
    public LocalDateTime getTimestamp() {
        return timestamp;
    }
    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }
}

```

```

@Override
public String toString() {
    return "CryptoCurrencyQuote{" +
        "currency='" + currency + '\'' +
        ", price=" + price +
        ", timestamp=" + timestamp +
        '}';
}
}

```

CryptoCurrencyService.java

```

package com.advanced_java.HrodskaM.service;
import com.advanced_java.HrodskaM.dto.CryptoCurrencyQuote;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import org.springframework.web.reactive.function.client.WebClient;
import java.time.Instant;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
@Service
public class CryptoCurrencyService {
    private final RestTemplate restTemplate;
    private final ObjectMapper objectMapper; // Для парсингу JSON
    public CryptoCurrencyService(RestTemplate restTemplate, ObjectMapper
objectMapper) {
        this.restTemplate = restTemplate;
        this.objectMapper = objectMapper;
    }
    // Get current Bitcoin rate in UAH
    public String getBitcoinRate() {
        String url =
"https://api.coingecko.com/api/v3/simple/price?ids=bitcoin&vs_currencies=usd";
        String response = restTemplate.getForObject(url, String.class);
        return extractBitcoinRate(response);
    }
    // Використовуємо Jackson для парсингу JSON
    private String extractBitcoinRate(String response) {
        try {
            JsonNode rootNode = objectMapper.readTree(response);
            JsonNode bitcoinNode = rootNode.path("bitcoin");
            if (bitcoinNode != null && bitcoinNode.has("usd")) {
                return "Bitcoin rate: " + bitcoinNode.path("usd").asText() + "
USDT";
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return "It was not possible to get rate.";
    }
    // Пошук криптовалюти за назвою
    private final WebClient webClient =
WebClient.create("https://api.coingecko.com/api/v3");

    public List<CryptoCurrencyQuote> getQuotes(String name) {
        List<CryptoCurrencyQuote> quotes = new ArrayList<>();
        ObjectMapper objectMapper = new ObjectMapper();
    }

```

```

        // беремо ціни за 30 днів
        var response = webClient.get()
            .uri("/coins/" + name +
                "/market_chart?vs_currency=usd&days=30&interval=daily")
            .retrieve()
            .bodyToMono(String.class)
            .block();

        try {
            // парсимо JSON відповідь
            JsonNode rootNode = objectMapper.readTree(response);
            JsonNode pricesNode = rootNode.path("prices");
            // перетворюємо кожен елемент JSON в CryptoCurrencyQuote
            for (JsonNode priceNode : pricesNode) {
                long timestamp = priceNode.get(0).asLong();
                double price = priceNode.get(1).asDouble();
                LocalDateTime date =
                    Instant.ofEpochMilli(timestamp).atZone(java.time.ZoneId.systemDefault()).toLocal
                        DateTime();
                quotes.add(new CryptoCurrencyQuote(name, price, date));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return quotes;
    }
}

```

ExcelService.java

```

package com.advanced_java.HrodskaM.service;
import com.advanced_java.HrodskaM.dto.CryptoCurrencyQuote;
import jakarta.servlet.ServletOutputStream;
import jakarta.servlet.http.HttpServletRequestResponse;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.springframework.stereotype.Service;
import java.time.format.DateTimeFormatter;
import java.util.List;
@Service
public class ExcelService {
    public void exportToExcel(List<CryptoCurrencyQuote> quotes,
        HttpServletResponse response) throws Exception {
        Workbook workbook = new XSSFWorkbook();
        Sheet sheet = workbook.createSheet("Quotes");
        Row header = sheet.createRow(0);
        header.createCell(0).setCellValue("Currency");
        header.createCell(1).setCellValue("Price (USD)");
        header.createCell(2).setCellValue("Timestamp");
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd
            HH:mm:ss");
        int rowIdx = 1;
        for (CryptoCurrencyQuote quote : quotes) {
            Row row = sheet.createRow(rowIdx++);
            row.createCell(0).setCellValue(quote.getCurrency());
            row.createCell(1).setCellValue(quote.getPrice());
            row.createCell(2).setCellValue(quote.getTimestamp().format(formatter));
        }
    }
}

```

```

    }
    response.setContentType("application/vnd.openxmlformats-officedocument.spreadsheetml.sheet");
    response.setHeader("Content-Disposition", "attachment; filename=quotes.xlsx");
    ServletOutputStream outputStream = response.getOutputStream();
    workbook.write(outputStream);
    workbook.close();
    outputStream.close();
}
}

```

index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HrodskaMariia Project</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <link rel="stylesheet" href="/css/style.css">
</head>
<body>
<h1> Hrodska JavaProject</h1>
<div>
  <h2><input checked="" type="checkbox"/> Check Bitcoin Rate</h2>
  <button id="get-bitcoin-rate">Show Current Bitcoin Rate</button>
  <p><a id="bitcoin-rate"></a></p>
</div>
<hr>
<div>
  <h2><img alt="Bitcoin icon" data-bbox="218 555 235 568"/> Search Cryptocurrencies</h2>
  <input type="text" id="crypto-search-query" placeholder="Enter cryptocurrency name...">
  <button id="parse-crypto"><img alt="Search icon" data-bbox="438 598 455 615"/> Search</button>
  <button id="download-excel" style="display: none; margin-top: 10px;"><img alt="Excel icon" data-bbox="858 615 875 632"/>
  Download Excel File</button>
  <!-- Таблиця для результатів -->
  <table id="crypto-table" style="display: none; margin-top: 20px; border-collapse: collapse; width: 100%;">
    <thead>
      <tr>
        <th style="border: 1px solid #ccc; padding: 8px;">Currency</th>
        <th style="border: 1px solid #ccc; padding: 8px;">Price (USD)</th>
        <th style="border: 1px solid #ccc; padding: 8px;">Date</th>
        <th style="border: 1px solid #ccc; padding: 8px;">Time</th>
      </tr>
    </thead>
    <tbody id="crypto-items"></tbody>
  </table>
</div>
<script>
  $(document).ready(function() {
    // Отримати курс біткоїна
    $('#get-bitcoin-rate').click(function() {
      $.get('/get-bitcoin-rate', function(data) {

```

```

        $('#bitcoin-rate').text(data);
    });
});
// Пошук криптовалют за назвою
$('#parse-crypto').click(function () {
    const searchQuery = $('#crypto-search-query').val();

    $.get('/parse-crypto', { searchQuery: searchQuery }, function (data)
    {
        const $table = $('#crypto-table');
        const $tbody = $('#crypto-items');
        const $downloadBtn = $('#download-excel');
        $tbody.empty(); // очищаємо старі дані
        if (data.length === 0) {
            $table.hide();
            $downloadBtn.hide();
            $tbody.append('<tr><td colspan="4">No cryptocurrencies
found</td></tr>');
        } else {
            data.forEach(function (item) {
                const formattedDate = new
Date(item.timestamp).toLocaleDateString('uk-UA', {
                    year: 'numeric',
                    month: 'long',
                    day: 'numeric',
                });
                const formattedTime = new
Date(item.timestamp).toLocaleTimeString('uk-UA', {
                    hour: '2-digit',
                    minute: '2-digit',
                });
                $tbody.append(`
                    <tr>
                        <td style="border: 1px solid #ccc; padding:
8px;">${item.currency.toUpperCase()}</td>
                        <td style="border: 1px solid #ccc; padding:
8px;">${item.price.toFixed(2)}</td>
                        <td style="border: 1px solid #ccc; padding:
8px;">${formattedDate}</td>
                        <td style="border: 1px solid #ccc; padding:
8px;">${formattedTime}</td>
                    </tr>
                `);
            });
            $table.show();
            $downloadBtn.show();
        }
    });
});
// Завантаження Excel
$('#download-excel').click(function () {
    const searchQuery = $('#crypto-search-query').val();
    window.location.href = '/download?name=' +
encodeURIComponent(searchQuery);
});
});
</script>
</body>
</html>

```


style.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #f4f6f8;
  color: #333;
  margin: 0;
  padding: 20px;
}
h1 {
  text-align: center;
  font-size: 2.5rem;
  margin-bottom: 30px;
  background: linear-gradient(90deg, #287dc9, #4251f5, #1ba897);
  background-size: 200% auto;
  color: transparent;
  background-clip: text;
  -webkit-background-clip: text;
  animation: gradientMove 4s linear infinite;
}
@keyframes gradientMove {
  0% {
    background-position: 0% center;
  }
  100% {
    background-position: 200% center;
  }
}
h2 {
  color: #333;
  font-size: 24px;
  margin-bottom: 15px;
  text-align: left;
}
button {
  background: linear-gradient(45deg, #0e4a80, #4251f5);
  color: white;
  padding: 10px 18px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 15px;
  transition: background 0.6s ease, transform 0.2s;
  margin-top: 10px;
  background-size: 200% 200%;
}
button:hover {
  background-position: right center;
  transform: scale(1.02);
}
input[type="text"] {
```

```

width: 100%;
padding: 10px;
font-size: 16px;
border: 2px solid #ddd;
border-radius: 5px;
margin-bottom: 20px;
transition: border-color 0.3s;
}
input[type="text"]:focus {
border-color: #4a90e2;
outline: none;
}
/* Таблиця */
table {
width: 100%;
border-collapse: collapse;
margin-top: 15px;
}
th, td {
border: 1px solid #ccc;
padding: 10px;
text-align: left;
}
th {
background-color: #e3f2fd;
color: #0d47a1;
}
/* Посилання */
a#bitcoin-rate {
font-weight: bold;
color: #43a047;
font-size: 18px;
display: inline-block;
margin-top: 10px;
}
hr {
border: 0;
border-top: 1px solid #ddd;
margin: 30px 0;
}
/* Responsive Design */
@media (max-width: 768px) {
body {
padding: 10px;
}
input[type="text"] {
font-size: 14px;
}
button {
font-size: 14px;
padding: 8px 15px;
}
}

```